



EAST TEXAS A&M
— UNIVERSITY —

CSCI 380.61E WEB PROGRAMMING AND INTERFACE

AVASCRIPT ES6+ (2021-2024)

- Arrays in ES6+
 - Declaring arrays
 - New array methods
 - Searching, sorting, and iterating arrays.
- Loops and Control Flow
 - for, for...in, for...of, while
 - break, continue, and other control flow mechanisms.
 - if...else and switch conditions.
- Regular Expressions

ES6 AND BEYOND

- Performance Improvements: The V8 engine and other JavaScript engines have optimized features like `async/await` for concurrency.
- Cleaner Syntax: Reduces boilerplate code with arrow functions, destructuring, and spread/rest operators.
- Modern JavaScript Features: ECMAScript has been evolving with every yearly update (e.g., ES2021, ES2022, ES2023, and ES2024) to keep up with the demands of modern web development.

ARRAYS IN JAVASCRIPT

- Arrays are ordered collections of values that can be of any data type.
- Arrays in JavaScript are dynamic and can grow or shrink in size.
- Common operations include adding/removing elements, sorting, filtering, and iterating.

ARRAYS IN JAVASCRIPT

- Declaring Arrays
 - Basic Declaration: `let fruits = ['apple', 'banana', 'orange'];`
 - Type: Array of strings.
 - Empty Array: `let numbers = [];`
 - Using `Array.of()` (ES6): `let numbers = Array.of(1, 2, 3);`
 - Using `Array.from()` (ES6+):
 - Creates an array from an iterable or array-like object.

```
let str = 'hello';
```

```
let arr = Array.from(str); // ['h', 'e', 'l', 'l', 'o']
```

DECLARING ARRAYS IN ES6+

- `Array.of()`:
- Syntax: `Array.of(element1, element2, ..., elementN)`
 - Converts a list of values into an array.

```
let arr = Array.of(10, 20, 30); // [10, 20, 30]
```
- `Array.from()`:
- Syntax: `Array.from(arrayLike)`
 - Converts iterables (like strings) or array-like objects (like arguments) into an array.

```
let arr = Array.from('hello'); // ['h', 'e', 'l', 'l', 'o']
```

ARRAY METHODS (ES6+)

- Common Array Methods:

- `push()`:

- Adds one or more elements to the end of an array.

```
let arr = [1, 2];
```

```
arr.push(3); // [1, 2, 3]
```

- `pop()`:

- Removes the last element of an array.

```
arr.pop(); // [1, 2]
```

ARRAY METHODS (ES6+)

- `shift()`:
 - Removes the first element from an array.
`arr.shift(); // [2, 3]`
- `unshift()`:
 - Adds one or more elements to the beginning of an array.
`arr.unshift(0); // [0, 1, 2]`

ARRAY METHODS (ES6+)

- `concat()`:
 - Merges two or more arrays.

```
let arr1 = [1, 2];
```

```
let arr2 = [3, 4];
```

```
let arr3 = arr1.concat(arr2); // [1, 2, 3, 4]
```

ARRAY METHODS (ES6+)

- `slice()`:

- Returns a shallow copy of a portion of an array.

```
let arr = [1, 2, 3, 4];
```

```
let newArr = arr.slice(1, 3); // [2, 3]
```

- `splice()`:

- Adds/removes items from an array at a specific index.

```
let arr = [1, 2, 3, 4];
```

```
arr.splice(1, 2, 'a', 'b'); // [1, 'a', 'b', 4]
```

MODERN ARRAY METHODS (ES6+)

- `map()`:

- Creates a new array with the results of calling a function for every array element.

```
const nums = [1, 2, 3];
```

```
const doubled = nums.map(num => num * 2); // [2, 4, 6]
```

- `filter()`:

- Creates a new array with all elements that pass the test implemented by the provided function.

```
const nums = [1, 2, 3, 4, 5];
```

```
const evenNums = nums.filter(num => num % 2 === 0); // [2, 4]
```

MODERN ARRAY METHODS (ES6+)

- `reduce()`:
 - Applies a function against an accumulator and each element in the array to reduce it to a single value.

```
const nums = [1, 2, 3];
```

```
const sum = nums.reduce((acc, num) => acc + num, 0); // 6
```

- `find()`:
 - Returns the first element in the array that satisfies the provided testing function.

```
const nums = [5, 10, 15];
```

```
const found = nums.find(num => num > 8); // 10
```

MODERN ARRAY METHODS (ES6+)

- `findIndex()`:
 - Returns the index of the first element in the array that satisfies the provided function.

```
const nums = [5, 10, 15];
```

```
const index = nums.findIndex(num => num > 8); // 1
```

ARRAY SEARCH METHODS

- `includes()`:
 - Checks if the array contains a specific element.

```
const nums = [1, 2, 3];
```

```
console.log(nums.includes(2)); // true
```

```
console.log(nums.includes(4)); // false
```

ARRAY SEARCH METHODS

- `indexOf()`:
 - Returns the index of the first occurrence of the specified element.

```
const nums = [1, 2, 3];
```

```
console.log(nums.indexOf(2)); // 1
```

```
console.log(nums.indexOf(4)); // -1
```

ARRAY SORTING

- `sort()`:
 - Sorts elements in place. By default, sorts lexicographically (for strings).

```
const nums = [10, 3, 5];  
nums.sort(); // [3, 5, 10]
```

- Sorting Numbers:
 - Use a comparator function to sort numerically.

```
const nums = [10, 3, 5];  
nums.sort((a, b) => a - b); // [3, 5, 10]
```

ARRAY SORTING

- Sorting Strings:
 - Lexicographical sorting is default.

```
const names = ['Bob', 'Alice', 'Charlie'];  
names.sort(); // ['Alice', 'Bob', 'Charlie']
```

ARRAY ITERATION METHODS

- `forEach()`:
 - Executes a provided function once for each array element.

```
const nums = [1, 2, 3];  
nums.forEach(num => console.log(num));
```

- `for...of`:
 - Iterates over the values of an iterable object.

```
const nums = [1, 2, 3];  
for (const num of nums) {  
  console.log(num);  
}
```

ARRAY ITERATION METHODS

- for...in:
 - Iterates over the keys (indexes) of an object or array.

```
const nums = [1, 2, 3];  
for (const index in nums) {  
  console.log(index); // '0', '1', '2'  
}
```

LOOPS IN JAVASCRIPT

- Basic for Loop:

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

while Loop:

```
let i = 0;  
while (i < 5) {  
  console.log(i);  
  i++;  
}
```

BREAK

- break:

- Exits a loop early.

```
for (let i = 0; i < 5; i++) {  
  if (i === 3) break; // Breaks the loop  
  console.log(i);  
}
```

CONTINUE

- continue:
 - Skips the current iteration and moves to the next iteration.

```
for (let i = 0; i < 5; i++) {  
  if (i === 3) continue; // Skips 3  
  console.log(i);  
}
```

CONDITIONAL STATEMENTS (IF, ELSE, SWITCH)

- if...else:

```
const age = 18;  
if (age >= 18) {  
  console.log('Adult');  
} else {  
  console.log('Minor');  
}
```

CONDITIONAL STATEMENTS (IF, ELSE, SWITCH)

- switch:

```
const day = 'Monday';  
switch (day) {  
  case 'Monday':  
    console.log('Start of the week');  
    break;  
  case 'Friday':  
    console.log('End of the week');  
    break;  
  default:  
    console.log('Midweek');  
}
```

COMPARISON OPERATORS

- Loose Equality (==): Compares values, ignoring data types.
- Strict Equality (===): Compares both values and types.

```
console.log(5 == '5'); // true (loose equality)
```

```
console.log(5 === '5'); // false (strict equality)
```

REGULAR EXPRESSIONS (REGEX)

- Basic Syntax:
 - /pattern/ or new RegExp('pattern')
 - Matching a pattern:

```
const regex = /\d+/  
console.log(regex.test("123")); // true
```

REGULAR EXPRESSIONS (REGEX)

- Common Regex Methods:
- `test()`: Tests if the regex pattern matches the string.
- `exec()`: Returns an array of matches.
- `replace()`: Replaces parts of the string matching the regex.

```
const regex = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;  
console.log(regex.test("test@example.com")); // true
```

JAVASCRIPT SCOPE AND HOISTING

- Scope determines where variables and functions can be accessed in the program.
 - Global Scope: Variables declared outside any function are globally accessible.
 - Function Scope: Variables declared inside a function are accessible only within that function.
 - Block Scope (ES6): Variables declared with `let` and `const` inside a block `{}` are limited to that block.

JAVASCRIPT SCOPE AND HOISTING

- Hoisting in JavaScript
- Hoisting refers to the behavior where variable and function declarations are moved to the top of their containing scope during compilation.

- var: Hoisted and initialized with undefined.

```
console.log(a); // undefined
```

```
var a = 5;
```

SUMMARY

- Arrays in ES6+ are more powerful with new methods for iteration, transformation, and searching.
- Loops (for, while, for...in, for...of) provide various ways to iterate over arrays and objects.
- if...else, switch, and comparison operators allow us to control the flow based on conditions.
- Regular expressions (regex) help in pattern matching and string manipulation.

JAVASCRIPT SCOPE AND HOISTING

- let and const: Hoisted but not initialized. ReferenceError occurs if accessed before initialization.

```
console.log(a); // ReferenceError: Cannot access 'a' before  
initialization
```

```
let a = 5;
```

- Function declarations are hoisted in their entirety.

```
foo(); // Works fine!
```

```
function foo() {  
  console.log('Hello!');
```

```
}  
31
```

VARIABLE DECLARATION SUMMARY

- **var**: Function-scoped, hoisted with initialization as undefined.
- **let**: Block-scoped, hoisted but not initialized.
- **const**: Block-scoped, hoisted but not initialized, must be assigned during declaration.

STRICT MODE

- Strict Mode enforces a stricter set of rules in JavaScript, helping to catch errors earlier.
- You can enable strict mode by adding "use strict"; at the beginning of a script or a function.

STRICT MODE

- Benefits of Strict Mode:
- Prevents the use of undeclared variables.
- Eliminates some silent errors.
- Disallows duplicate property names or parameters.

"use strict";

x = 5; // ReferenceError: x is not defined

THE THIS KEYWORD IN JAVASCRIPT

- this refers to the context in which a function is called. It's a reference to the object that is currently invoking the function.
- How this Works:
- In Global Scope:
 - In non-strict mode, this refers to the global object (window in browsers).

`console.log(this); // In browsers: window`

THE THIS KEYWORD IN JAVASCRIPT

- Inside a Function:
- In regular functions, this refers to the object that called the function.

```
function greet() {  
    console.log(this); // `this` refers to the object calling the function  
}
```

`greet();` // In global scope (non-strict mode), `this` will refer to window.

THE THIS KEYWORD IN JAVASCRIPT

- Inside an Object Method:
- `this` refers to the object that owns the method.

```
const person = {  
  name: "Alice",  
  greet: function() {  
    console.log(this.name); // "Alice"  
  }  
};  
  
person.greet();
```

THE THIS KEYWORD IN JAVASCRIPT

- In Arrow Functions:
- Arrow functions do **not** have their own this but inherit this from the surrounding scope.

```
const person = {  
  name: 'Alice',  
  greet: () => {  
    console.log(this.name); // undefined, as `this` refers to global scope  
  }  
};  
person.greet();
```

THE THIS KEYWORD IN JAVASCRIPT

- In Classes:
- Inside class methods, this refers to the class instance.

```
class Person {  
  constructor(name) { this.name = name; }  
  greet() { console.log(this.name); }  
}
```

```
const p = new Person('Alice');  
p.greet(); // "Alice"
```

CLASSES IN ES6+

- Classes are syntactical sugar over JavaScript's existing prototype-based inheritance.
- They provide a clearer, more object-oriented approach to defining blueprints for objects.
- Defining a Class:

```
class Person {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
}
```

CLASSES IN ES6+

```
greet() {  
  console.log(`Hello, my name is ${this.name} and I am ${this.age} years  
old.`);  
}  
}
```

```
const p = new Person('Alice', 30);  
p.greet(); // Hello, my name is Alice and I am 30 years old.
```

CLASSES IN ES6+

- Inheritance with Classes:
- Classes can **inherit** from other classes using the extends keyword.

```
class Employee extends Person {  
  constructor(name, age, position) {  
    super(name, age);  
    this.position = position;  
  }  
}
```

CLASSES IN ES6+

```
describe() {  
  console.log(` ${this.name} works as a ${this.position}.` );  
}  
}
```

```
const e = new Employee('Bob', 25, 'Developer');  
e.describe(); // Bob works as a Developer.
```

JSON IN JAVASCRIPT

- What is JSON?
- JSON (JavaScript Object Notation) is a lightweight data interchange format that's easy for humans to read and write, and easy for machines to parse and generate.
- Working with JSON:
 - Parsing JSON (from string to object):

```
const jsonStr = '{"name": "Alice", "age": 25}';  
const obj = JSON.parse(jsonStr);  
console.log(obj); // {name: "Alice", age: 25}
```

JSON IN JAVASCRIPT

- Stringifying an Object (from object to JSON string):

```
const person = { name: "Bob", age: 30 };
```

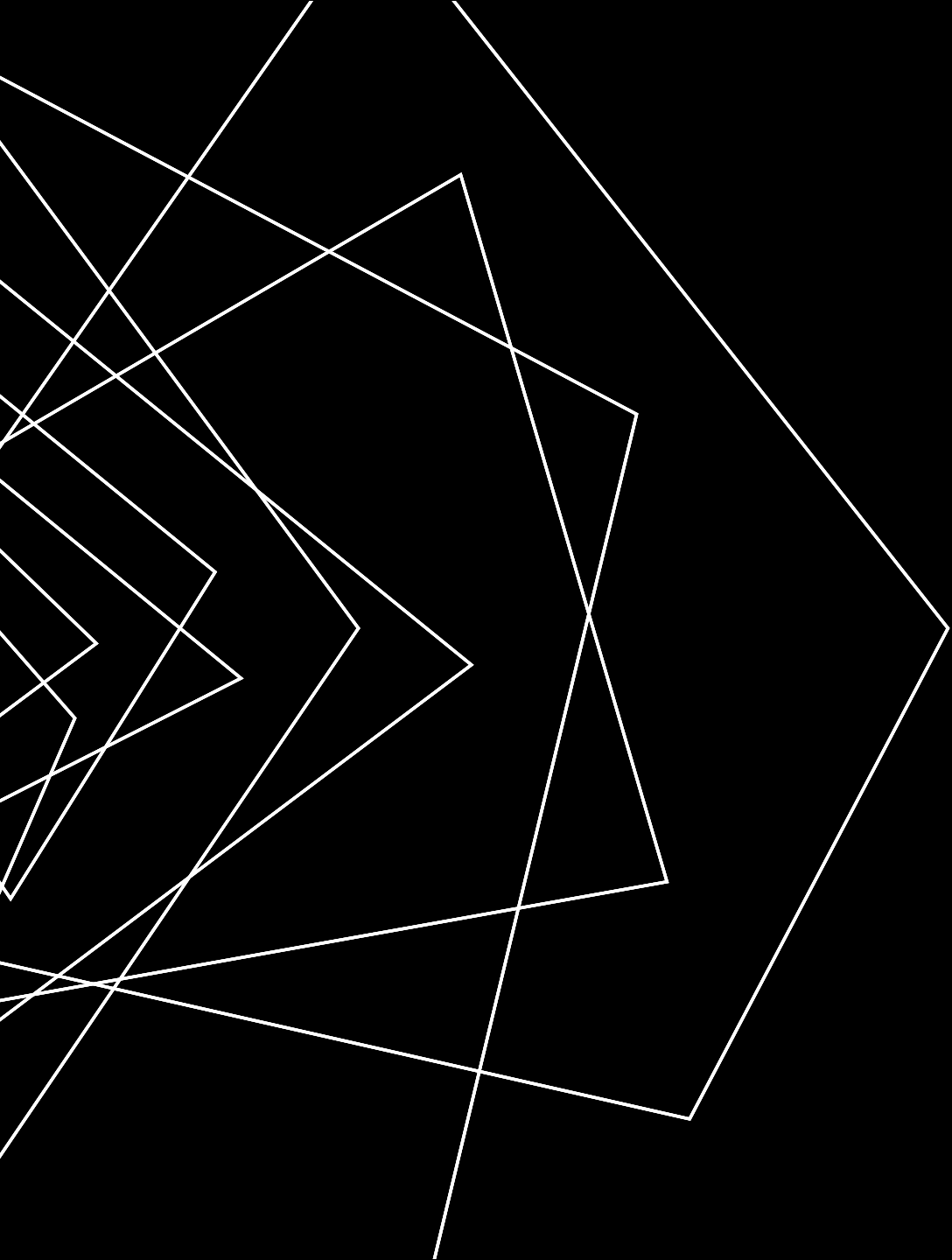
```
const jsonStr = JSON.stringify(person);
```

```
console.log(jsonStr); // '{"name": "Bob", "age": 30}'
```

JSON IN JAVASCRIPT

- Use Case: Fetching Data from an API:
- Often, data returned from web APIs is in JSON format. You can parse it to use in your application.

```
fetch('https://api.example.com/user')  
  .then(response => response.json())  
  .then(data => console.log(data));
```



THANK YOU

Yau Ka Cheung
Yauka.Cheung@etamu.edu
ACB1 Room 335